MLOps Learning Journey 🚀

A structured, hands-on roadmap from Python fundamentals to a deployable MLOps project.
Built from real Kaggle notebooks — every example was written and tested in a live environment.

👤 About

I'm transitioning into MLOps and Data Science, documenting my learning publicly to build a practical portfolio.
This repo tracks my progress week by week — from NumPy basics to a fully deployed ML pipeline.

🗺️ Roadmap

Week	Topic	Status
1–2	NumPy — data layer foundations	✅ Complete
3–4	Pandas — structured data & cleaning	✅ Complete
5	scikit-learn — preprocessing, scaling, class imbalance & model tuning	✅ Complete
6	scikit-learn — QuantileTransformer, GridSearchCV, Pipelines	✅ Complete
7	scikit-learn — VotingClassifier (soft voting, weighted estimators)	✅ Complete
8	scikit-learn — decision thresholds, StackingClassifier (meta-model)	✅ Complete
9	scikit-learn — grouped models (per-category), time decay (scikit-lego)	✅ Complete
10	FastAPI — handlers, async, multiple servers, Pydantic request logs, JSON auto-serialization, HTML forms, CRUD (wrapping a model in an API)	✅ Complete
11	FastAPI — raw request lookup by ID, system stats endpoint, start/stop snapshots, `response_model` (response shaping/security), real deployment + debugging in PyCharm; VM start/stop monitoring (RFC, Compute Engine scoped)	🔄 In Progress

Week	Topic	Status
11b	scikit-learn — `human-learn`: rule-based `FunctionClassifier` (Titanic fare threshold), `FunctionOutlierDetector` for outlier detection (penguin dataset)	✅ Complete
12	Docker — containerise the API	⏳ Upcoming
13	CI/CD + Deployment — GitHub Actions + Render/Railway	⏳ Upcoming

📚 Course Notes

Self-written beginner guides with tester, data analyst, and MLOps perspectives on every topic.

File	Description
`numpy_mlops_course.md`	Array creation, reshaping, boolean filtering, broadcasting, save/load + mini project
`pandas_mlops_course.md`	DataFrames, null handling, groupby, merging, encoding + ML-ready pipeline project
`categorical-encoding-notes.md`	OrdinalEncoder vs OneHotEncoder, rare/unknown category handling, `min_frequency` vs `max_categories`, NumPy slicing + `plt.scatter` refresher
`scaling-and-model-selection-notes.md`	QuantileTransformer (non-linear, rank/percentile-based scaling, robust to outliers vs StandardScaler), GridSearchCV (hyperparameter search + cross-validation against a scoring metric), model selection reasoning for Linear Regression
`gridsearchcv-parameters-notes.md`	Full GridSearchCV parameter reference: `estimator`, `param_grid`, `cv`, `n_jobs`, `refit`, `verbose`, plus `scoring`, `error_score`, `return_train_score`, `pre_dispatch`
`sample-weight-and-numpy-notes.md`	sample_weight vs. class_weight (per-row vs. per-category importance), `make_scorer` for custom metrics inside GridSearchCV, `np.linspace`, `np.log` / `np.log1p`
`voting-classifier-notes.md`	VotingClassifier — combining multiple estimators, `voting="soft"` (averaged predicted probabilities) vs `voting="hard"` (majority vote), `weights` parameter for weighting each estimator's influence
`threshold-in-scikit-learn.md`	Decision thresholds: `predict_proba` vs `decision_function` defaults, precision/recall trade-off, `precision_recall_curve`,

File	Description
	`roc_curve`, `TunedThresholdClassifierCV`
`threshold-and-metamodel-notes.md`	Applying decision thresholds on top of a `StackingClassifier` meta-model; meta-model (learns to combine base models) vs. VotingClassifier (fixed, hand-set weights)
`grouped-models-time-decay-notes.md`	One model per category via manual splitting or `GroupedPredictor` (scikit-lego); time decay via `sample_weight` or `DecayEstimator` (scikit-lego); grouping vs. time decay as distinct, combinable strategies; train/test distribution-shift risk
`fastapi-handlers-async-crud-notes.md` *(updated)*	FastAPI basics: handlers as functions, `asyncio.sleep` vs `time.sleep` in async routes, running multiple uvicorn servers, Pydantic request-log schema, automatic JSON request/response serialization, HTML forms (`Form(...)`) for server + front-end data capture, CRUD ↔ HTTP methods, running in Kaggle and Google Cloud Shell
`fastapi-traceability-matrix.md` *(updated)*	Traceability matrix mapping each FastAPI concept to its implementation and test status, for future reference
`fastapi-rfc-vm-monitoring.md` *(updated)*	RFC design doc for starting/stopping a cloud VM and reading its stats via FastAPI, scoped against the Google Cloud Compute Engine API — design not yet implemented; includes a progress log of related real-deployment groundwork (PyCharm, `response_model`, GET/POST testing)
`sklearn-rfc-humanlearn-outliers.md` *(new)*	`human-learn` outlier detection resolved: `FunctionOutlierDetector` / `InteractiveOutlierDetector`, scikit-learn-style `1` / `-1` API, drawing polygons on charts with custom group labels, the `threshold` parameter and missing-feature handling, plus a local-environment assignment using the penguin dataset

🗂️ Repo Structure

```
MLOps beginner friendly learning
│
├── README.md
│
├── courses/
│   ├── numpy_mlops_course.md
│   ├── pandas_mlops_course.md
│   ├── categorical-encoding-notes.md
│   ├── scaling-and-model-selection-notes.md
│   ├── gridsearchcv-parameters-notes.md
```

```
│   ├── sample-weight-and-numpy-notes.md
│   ├── voting-classifier-notes.md
│   ├── threshold-in-scikit-learn.md
│   ├── threshold-and-metamodel-notes.md
│   ├── grouped-models-time-decay-notes.md
│   ├── fastapi-handlers-async-crud-notes.md   (updated)
│   ├── fastapi-traceability-matrix.md   (updated)
│   ├── fastapi-rfc-vm-monitoring.md   (updated)
│   └── sklearn-rfc-humanlearn-outliers.md   (new)
│
├── notebooks/
│   ├── numpy_practice.ipynb
│   ├── pandas_practice.ipynb
│   └── scikit-learn practice → see Kaggle link below
│
└── projects/
    └── (deployable project coming in Week 12)
```

🛠️ Tools & Environment

- Kaggle Notebooks — free GPU, pre-installed libraries
- Google Cloud Shell — running and testing FastAPI servers
- PyCharm — running and debugging FastAPI locally (real GET/POST testing via `/docs`, not simulated)
- Python 3 — NumPy, Pandas, scikit-learn, scikit-lego, FastAPI, uvicorn, httpx
- Docker — containerisation
- GitHub Actions — CI/CD automation

📓 Kaggle Notebooks

- scikit-learn — preprocessing, encoding, scaling, class imbalance handling (sample_weight/class_weight), hyperparameter tuning with GridSearchCV, VotingClassifier (soft voting, weighted estimators), decision thresholds & StackingClassifier meta-models, grouped models per category & time decay (scikit-lego), `human-learn` rule-based `FunctionClassifier` and `FunctionOutlierDetector`
  🔗 https://www.kaggle.com/code/kannammaivr/scikit-learn
- FastAPI — handlers, async vs blocking sleep, multiple uvicorn servers on different ports (run in a background thread inside the notebook), Pydantic request-log schema, automatic JSON serialization, HTML forms for survey-style data capture, raw request lookup by unique ID, system stats via `psutil`, start/stop snapshots, `response_model` for response shaping/security, CRUD basics
  🔗 https://www.kaggle.com/code/kannammaivr/fastapi-html-json

🔗 Connect

LinkedIn: https://www.linkedin.com/in/kannammai-v-r-03a498239

Updated weekly. Follow along if you're on a similar journey.